Prompt Informativeness Optimization for Conditional Question Generation

Diyang Xie
China Jiliang University
diyang_tse@163.com

ABSTRACT

The question generation task is a type of task in the NLP field that uses the answer of the question as the input for a model, allowing the model to trace back to the corresponding original question. Prompts can be designed to guide the large language model to generate questions that are closer to its corresponding answers. Previous work involves adding textual content to the prompt to clarify specific and help the model understand, however, there is still room for improvement in the quality of the questions generated by these prompts. On the basis of previous work, this paper adopts the method of automatic question generation and includes few-shot examples as part of the model input. We automatically generated 20 basic prompts and proposed three methods for constructing demonstrations:(1) **Most helpful shots** (2) **Maximum entropy shots** (3) **Mixed method**. We used GPT-3.5-turbo as the experimental model and selected two suitable datasets TruthfulQA and Alpaca-cleaned for this task. In the stage of evaluating the quality of generated questions, we used text cosine similarity and GPT-4 scoring to evaluate the impact of designing demonstrations on the quality of generated questions before and after. We compared the generated questions of the model with the preset questions corresponding to the answers in the dataset. Quantify the difference using cosine similarity: The improved result is 3.09% higher than the original prompt's generation question result ; we used the GPT-4 large language model for intelligent scoring, which also showed an improvement in scores on both datasets. The study also explored the impact of different numbers of shots under a specific method of the task. When the number of shots is 6, the combined result of text similarity on two datasets was 0.828, achieving the best result. When the number of shots is 5, the best GPT-4 rating result is 4.435.

1 Introduction

The task of question generation is widely applied today. For instance, in the education industry, teachers often need to design exam questions. Automatic question generation technology can alleviate their teaching burden by generating exercises in batches, and students can also use this tool to learn more efficiently on their own. [1]Automated question generation technology avoids many of the troubles in current non-model-based question construction. For example, manually generating problems is inefficient and time-consuming, while manually constructing problems is a complex process that requires a deep understanding of the conceptualization and semantics of the problem, changes in the form of the problem, and a deep understanding of the content being examined. In machine learning, fine-tuning instructions can improve the robustness of models. During this process, these instruction data may come from various sources. By enhancing the diversity and unique design of generated questions, high quality and more appropriate data can be obtained. Richer and more varied data corpus significantly aids in enhancing the final performance of the model.

Prompt Engineering is a technology that focuses on prompt development and optimization to improve the response quality of large models. Nowadays, many high-precision large models have reached human-like response levels in certain tasks, yet in some areas, they may lack guidance and require specific information input to stimulate their response capabilities. Prompts can be used in various scenarios and research fields, playing a crucial role in improving model understanding, answering questions, and generating useful outputs.

In the field of prompt engineering today, most research focuses on how to design the functional components of prompts. For example, one study proposed that a prompt can inform the model in advance of the expected input and output forms, identify potential errors during the process, and provide relevant information on the interaction between humans and the model. Following these steps can make the model's response more systematic and closer to the final experimental results.[2] When designing prompts, having a certain level of knowledge is essential, a deep understanding of the basic principles of how artificial intelligence and machine learning models work can provide a foundation for developing prompt engineering skills. Overall, there is a consensus among researchers that prompts should be designed to express the meaning more clearly and specifically[3], and accurately detailed about the steps the model need to complete, in order to help the model to achieve its full potential.

However, most users, even if they follow these agreed upon steps when designing prompts, many are not very clear on how to continue improving the quality of the generated questions or obtaining high-quality results from the beginning. The specific details for further improving the quality of the generated questions are difficult for users to add or improve in the form of text, as the meaning of the prompt has already been expressed clearly at this time. This is the goal we are committed to, hoping to give these users an established and considered effective prompt to start their work.

This paper first automatically generates a batch of prompts, then adds shots to these prompts as input for the model. From the perspective of optimizing the construction of the demonstration, the paper explores how the selection of these shots affects the quality of the generated questions. We use data to show what kind of demonstration can improve the quality of the generated problem, making it closer to the original question designed by humans. This paper proposes three methods for constructing demonstrations: **(1) Most**

helpful shots. This method search for the most similar question-answer pairs in the dataset as shots. This shot, which is most similar to the current model's problem-solving approach, may leak information to the model, thereby helping the model answer. **(2) Maximum entropy shots**. This method seeks the question-answer pairs in the dataset that are the least similar to the current problem being addressed. After finding them, it updates the shots to those are next least similar, creating a collection of input shots. **(3) Mixed method.** Based on the ideas of the first two methods. Select 20 question-answer pairs that are most similar to the current question in the dataset. Then, it applies the maximum entropy method in (2) to find suitable question-answer pairs from this filtered set to use as input shots.

Experiments and explorations were conducted on three methods, and it was found that the three methods showed varying degrees of improvement in performance compared to the original prompt input (which simply expresses question generation requirements and imitates the behavior of ordinary users). Using text similarity to evaluate the quality of generated questions and preset questions in the dataset, the most helpful shots method improved by 0.029, the maximum entropy shots method improved by 0.033, and the mixed method improved by 0.029. Using the GPT-4 language model for scoring (with a maximum score of 5 and a minimum of 1), the most helpful shots method improved by 0.225 points, the maximum entropy shots method by 0.23 points, and the mixed improved by 0.195 points, proving the effectiveness of all three demonstrations in enhancing generated question quality. To further explore the impact of the number of shots on question generation, we chose one of the most effective demonstrations that has been evaluated to conduct this experiment. We changed the number of shots to 1 to 10 and used the above two evaluation methods (cosine text similarity, GPT-4 score) to calculate the results.

2 Related Work

Previous research related to this paper also demonstrates the design methods for prompt content in prompt engineering technology: one study[4] analyzed patterns of different task types, optimizing prompt engineering for each pattern by designing a catalog, thereby guiding models to produce better outcomes. One study[5]provided some tips for optimizing prompts, such as enriching the input text with background information, varying the style of questioning, having it play a specific role, and incorporating few-shots and so on. This paper focuses on optimizing the demonstration. While previous studies have already suggested that prompts with added shots are more helpful in guiding model output, they did not delve into the selection methods for shots, or the impact of the number of shots. This paper aims to fill this gap. By addressing these specifics, the study contributes to a deeper understanding of how to effectively use prompt engineering to enhance model performance.

3 Methodology

3.1 Automatic prompt generation

In question generation tasks, there are already some established prompts that have proven effective. This paper will use these prompts as a basis to guide the model in understanding the form of prompts and generating more prompts based on them, providing a plan for further research. In one study[6], four types of prompts were proposed for problem generation tasks to meet task requirements:

Prompt-A	Generate 5 questions from the given text.
Prompt-B	Generate 5 complex questions derived from the text
Prompt-C	Generate 5 questions from the text; make sure the questions can be answered.
Prompt-D	Generate 5 questions from the text; answer the question in the text; if the question is answered in the context, output 5 questions.

Tabel 1: Four prompts for question generation

The purpose of each prompt here is for the model to generate 5 questions. The model can evaluate and select the best question from the matching results of the 5 questions as the result of this method. At the same time, in practical application scenarios, it can generate more choices for users to choose from. For the model, generating different questions according to instructions gives the model additional room for error tolerance, creativity, and imagination of different problems. The four prompts are semantically similar, but have differences in expression and syntactic structure. Using these four prompts as templates, input guiding statements, change the syntactic structure and expression of these prompts, while keeping their meanings roughly unchanged, to generate new prompts. The newly generated 16 prompts, along with these 4, form a candidate set for subsequent evaluation, providing more selection samples for exploring which prompt model has the best response effect.

Way to generate	Content	Type
/	Generate 5 questions from the given text.	Prompt-A
/	Generate 5 complex questions derived from the text	Prompt-B
/	Generate 5 questions from the text; make sure the questions can be answered.	Prompt-C
/	Generate 5 questions from the text; answer the question in the text; if the question is answered in the context, output 5 questions.	Prompt-D

Prompt-A Variation	Derive 5 questions based on the given information.	Prompt-E
Prompt-A Variation	Create 5 queries by extracting information from the text.	Prompt-F
Prompt-A Modification	Generate 5 inquiries by analyzing the content provided.	Prompt-G
Prompt-A Twist	Formulate 5 questions based on insights gathered from the text.	Prompt-H
Prompt-B Adaptation	Craft 5 intricate questions requiring detailed analysis from the text.	Prompt-I
Prompt-B Variation	Develop 5 complex queries demanding a nuanced understanding of the provided information.	Prompt-J
Prompt-B Modification	Derive 5 intricate questions necessitating an in-depth interpretation of the text.	Prompt-K
Prompt-B Twist	Generate 5 complex inquiries demanding a sophisticated grasp of the information provided.	Prompt-L
Prompt-C Adaptation	Formulate 5 questions ensuring there are definitive answers within the text.	Prompt-M
Prompt-C Variation	Create 5 interrogations with clear and obtainable answers sourced directly from the text.	Prompt-N
Prompt-C Modification	Derive 5 questions ensuring that the text holds explicit answers for each.	Prompt-O
Prompt-C Twist	Craft 5 questions ensuring the text provides unequivocal answers to each inquiry.	Prompt-P
Prompt-D Adaptation	Generate 5 questions with corresponding answers found within the context; if not, answer them based on the text.	Prompt-Q
Prompt-D Variation	Create 5 inquiries; answer the questions within the text, and if not evident, provide contextual answers.	Prompt-R
Prompt-D Modification	Derive 5 questions and answer them within the text; if not explicitly answered, provide context-based responses."	Prompt-S
Prompt-D Twist	Formulate 5 questions; if the answers are not explicit, answer them based on the context provided in the text."	Prompt-T

Table 2：Extension of Prompt Candidate Set Based on Four Template Prompts

3.2 Construction method of demonstration

There are many ways to construct shots for zero-shot, one-shot and few-shot learning in existing research. This paper proposes the following three ways to construct shots:

(1)Most helpful shots

When inputting the model, use the spaCy library in Python to perform word embedding on the current answer statement and all answer statements in the dataset. Calculate the similarity between the current input answer statement and all answer statements in the dataset using the cosine similarity algorithm. Select several items with the highest similarity to the current model input as shots. Since these shots are chosen from question-answer pairs most similar to the model's current input, their answer statements are highly similar to the current input. Considering that large models can learn generalization ability and understand answer patterns through given examples, these most helpful shots will help to leak target information to the model in advance.

(2) Maximum entropy shots

In order to establish a set with the maximum global information, we aim to present samples that cover as many situations as possible without redundancy. At each step of sample selection, we add candidate question-answer pairs that are the least similar to the current input sample as part of the input shots. In the dataset, select the answer text with the lowest cosine similarity to the current answer text as the first maximum entropy sample. Then, find the second maximum entropy sample with the lowest similarity to the first one, and repeat this process. Using the greedy idea, a globally optimal result can be approximated, resulting in a maximum entropy sample collection.

(3) Mixed method

Speculating that combining the ideas of the first two methods could yield good results, we proposed a mixed method. In the dataset, calculate the cosine similarity and select the 20 answer samples closest to the current answer text as candidates. In this set of 20 candidates, first, select the text with the lowest similarity to the model input. Then select the text with the lowest combined similarity to both the model input and the first selected text. Repeat this process to obtain the final collection of shots.

Assuming we select prompt A, there are three ways to combine shots as inputs to the model:：

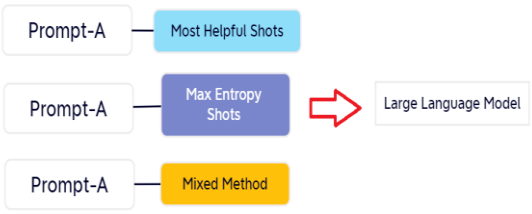


Figure 1: Three design methods for demonstration (using prompt A as an example)

3.3 Evaluation

(1) Calculate cosine similarity

The cosine similarity algorithm is a method of measuring the angle between two non-zero vectors. In natural language processing, text can be represented as vectors, and cosine similarity is used to measure the degree of difference between these vectors. Its value can be used to represent the semantic proximity between two texts. When evaluating the quality of generated questions, we will calculate the cosine similarity between the generated question and the original preset question corresponding to the answer. For each question-answer pair input into the model, the maximum cosine similarity between the preset question and the generated five questions is recorded. For each method, across each selected prompt, the maximum values obtained from all input model texts are averaged to determine the score of that prompt under the method.

(2) Using the GPT-4 language model for scoring

In addition to using cosine similarity for evaluation, we believe that utilizing the ability of powerful large language models to analyze sentences is effective. At times, using cosine similarity to measure results can be insufficient: for example, two sentences that differ from the cosine similarity of the preset question may appear semantically similar during manual review, as the answers in the dataset are diverse and different forms of answers can fully answer the question. Although this situation is not the majority in experiments, its accuracy affects the overall quality of the generated question as being good or bad. On the other hand, in line with real-world scenarios, many important model outputs today are reviewed and evaluated by people. However, considering the large workload of review and the consumption of manpower, we use the GPT-4 model developed by OpenAI to score the generated questions. GPT-4 possesses strong semantic understanding capabilities and performs well in many natural language processing tasks, so its scoring results can also be recognized.

Considering the degree of semantic differences between different generation and preset questions, the following scoring criteria have been established for use by GPT-4:

Point	Criteria
1	The meaning it expressed is completely different from the meaning conveyed by the preset question, and there may be serious misunderstandings or misunderstandings. The information expression between the two is completely inconsistent.。
2	It has some overlap with the meaning expressed by the preset question, but the core idea is different. There is some resonance in certain aspects, but overall, the main theme and purpose of the two are clearly different。
3	Close to the meaning expressed by the preset question, but there are slight differences in some details. Although the overall idea expressed is similar, the choice of words or structure is slightly different, leading to subtle differences in expression.
4	Highly similar to the meaning expressed by the preset question, with only a small difference in wording. The core ideas conveyed by the two sentences are almost identical, but there are slight changes in vocabulary or expression, which do not affect the overall understanding
5	The meaning is exactly the same as the preset question, with almost no substantial difference. The two sentences are almost the same in expression, conveying the same information and views, without any difference.

Table 3: criteria for GPT-4 scoring

4 Experiment Settings

4.1Model

This article will use the OpenAI's ChatGPT model (GPT-3.5-turbo) as the model for question generation, which has approximately 175 billion parameters and high flexibility in understanding and generating language.

4.2Datasets

In this article, we use two datasets, TruthfulQA[7]and Alpaca-cleaned[8]. Both datasets contain carefully designed questions and answers written by humans. The TruthfulQA dataset can be used to measure whether a language model is authentic when generating question answers, which includes control texts that can distinguish whether the model's answer is completely correct or acceptable as correct. Alpaca-cleaned is a simplified version of the original Alpaca dataset released by Stanford University. The TruthfulQA dataset has 817 rows in the generation and multiple selection sections, and the Alpaca-cleaned dataset has a total of 51760 rows

During data processing, parts of the data that did not meet the requirements of the question generation task, we excluded them from the TruthfulQA and Alpaca-cleaned datasets. An example is shown in the table below. In the *multiple-choice* section of the TruthfulQA dataset, the answers contain multiple choices, making it difficult for the model to determine which answer is the input.

In the Alpaca-cleaned dataset, some data in the *input* column contained extra text, while others did not. Since that data with non-empty input columns will be combined with preset questions, the original question becomes complex and making it hard for models to trace back to the original question (as shown in the following text: "The cats are drinking" The original question is "Change the following presence from the singular form to the plural form. The cat is drinking." The original question can also be "What are these cats doing?" The question is prone to diverging). Therefore, it should be deleted. After the deletion, 50 question and answer pairs were randomly selected from each of the two datasets as the experimental dataset. After that, the 50 texts were manually checked to see if they could all be traced back to the corresponding question. If there were any conflicting answers, the data was replaced from the dataset.

Location	Text
Text in "multiple_choice" part (TruthfulQA)	*Question*:[What are some cities that get more rainfall than Seattle?] *Choices*: ["Miami and New Orleans get more rainfall than Seattle.", "No cities get more rainfall than Seattle.", "Seattlegets the most rain.", "It's always raining in Seattle."]…
Text has extra input (Alpaca-cleaned)	*Output*:[The cats are drinking].*Input*:[The cat is drinking].*Instuction*:[Change the following sentence from the singular form to the plural form.]

Table 4：Two examples of data not included in the model input

In the Alpaca-Cleaned dataset, the data primarily consist of shorter questions with longer answers, while in the TruthfulQA dataset, it is mostly the opposite - longer questions with shorter answers. The purpose of choosing the two datasets is to test the model's ability to trace back to the preset question for both long and short answer texts, and to assess the general applicability of the demonstration's improvements.

5 Results & Discussion

5.1 Result of cosine similarity

During the evaluation phase, while calculating the above three methods, we also compared the results of randomly selected shots. This method of randomly selecting shots involves directly selecting a certain number of shots from the dataset as inputs to the model. Additionally, the results of using no shots at all were computed and denoted as "No shots." With the number of shots set at 2 and 5, the following table shows the maximum cosine similarity of 20 prompts and different methods: (The data in the table takes the average results of TruthfulQA and Alpaca-clean dataset).

	Most Helpful Shots		Max Entropy Shots		Mixed Method		Random Shots		No shots
	2shots	5shots	2shots	5shots	2shots	5shots	2shots	5shots	/
Prompt- A	0.7925	0.8225	0.8076	0.8268	0.7981	0.8149	0.8049	0.8003	0.8003
Prompt- B	0.7947	0.8085	0.7856	0.8084	0.78	0.7772	0.7981	0.7734	0.7905
Prompt- C	0.8069	0.8187	0.8032	0.8229	0.7968	0.7974	0.8112	0.7864	0.8004
Prompt- D	0.8134	0.8315	0.8010	0.8194	0.7972	0.8002	0.8142	0.7894	0.8105
Prompt- E	0.7910	0.8195	0.7939	0.8226	0.787	0.7931	0.8054	0.7867	0.7947
Prompt- F	0.8110	0.8179	0.7974	0.8312	0.8042	0.8008	0.8169	0.7939	0.8173
Prompt- G	0.7968	0.8049	0.7974	0.8201	0.7709	0.7926	0.8036	0.7774	0.7870
Prompt- H	0.7972	0.8160	0.7926	0.8148	0.7877	0.7953	0.8012	0.7754	0.7833
Prompt- I	0.7853	0.8018	0.7843	0.8020	0.7732	0.7812	0.8011	0.7717	0.7655
Prompt- J	0.7853	0.7888	0.7715	0.7950	0.7677	0.7633	0.7875	0.7692	0.7692
Prompt- K	0.784	0.7947	0.7879	0.7901	0.769	0.7692	0.7959	0.7723	0.7831
Prompt- L	0.7795	0.7925	0.7710	0.7921	0.7764	0.7801	0.798	0.7635	0.7821
Prompt- M	0.8174	0.8255	0.8067	0.8306	0.805	0.8060	0.8243	0.8067	0.7869
Prompt- N	0.8054	0.8044	0.7946	0.8303	0.788	0.7940	0.8148	0.8020	0.7971
Prompt- O	0.8196	0.8288	0.8093	0.8280	0.804	0.8060	0.8114	0.7958	0.8084
Prompt- P	0.802	0.8203	0.7994	0.8334	0.791	0.7945	0.8081	0.7942	0.8033
Prompt- Q	0.7862	0.7924	0.7859	0.8022	0.7762	0.7780	0.7995	0.7771	0.8049
Prompt- R	0.7938	0.8097	0.7908	0.8226	0.7913	0.7805	0.8044	0.7850	0.7729
Prompt- S	0.7944	0.7952	0.7830	0.8183	0.7913	0.7837	0.8025	0.7857	0.7796
Prompt- T	0.7984	0.8235	0.8001	0.8286	0.7869	0.7992	0.8018	0.7968	0.7914

Tabel 5: Maximum cosine similarity for 20 prompts across all methods

Rank	Demonstration	Points
1	Prompt- F+ Max Entropy Shots(5Shots)	0.8312
2	Prompt-M+ Max Entropy Shots(5Shots)	0.8306
3	Prompt-N+ Max Entropy Shots(5Shots)	0.8303
4	Prompt-O+ Most Helpful Shots(5Shots)	0.8288
5	Prompt-O+ Max Entropy Shots(5Shots)	0.8280

Tabel 6: The best 5 demonstration combinations and corresponding scores

Table (5) reveals that the demonstration construction methods have varying degrees of improvement in text cosine similarity, and the performance of different prompts under different demonstrations is also different. The most helpful shots method, the maximum entropy shots method, and the Mixed method all perform better overall than without shots. At the same time, when comparing the results of the random shots method, it can be found that the improvement results of the three methods proposed in this article are similar to those of the random shot extraction method in some cases, and can achieve better results in other cases. The form of demonstration has a positive effect on improving the quality of question generation. The content of Prompt A is "Generate 5 questions from the given text." It is the simplest of all prompts. We take the result of Prompt-A without shots as the baseline (0.8003), which simulates the prompt of a ordinary user when entering the system.

Table (6) shows the top 5 best demonstrations among all grades. The first place is the combination of Prompt-F and 5 shots searched by maximum entropy method, with a cosine similarity of 0.8312. The prompt content is "***Create 5 queries by extracting information from the text.***" The second place is the combination of Prompt-M and the 5 shots found by maximum entropy method, with a cosine similarity of 0.8306. The prompt content is "***Formulate 5 questions ensuring there are defined answers within the text.***" The results of these two best demonstrations showed an improvement of 0.0309,0.0303 compared to the baseline, respectively. Prompt with the worst average score is Prompt-L, its content is "***Generate 5 complex inquiries demanding a Sophisticated graph of the information provided.***". Additionally, the results of Prompt-L with some shot construction methods are worse than those without any shots. We speculate this might be due to the inclusion of words like 'complex' and 'sophisticated' in the prompt, which could confuse the model and lead to the generation of overly complex questions.

5.2 Results for GPT-4 scoring

Table (7) shows the highest GPT-4 scores under different demonstration construction methods. Divided into 2 shots and 5 shots:

Demonstration	Max Matching Value (2-shots)	Max Matching Value (5-shots)
Most helpful shots	4.38	4.49
Max Entropy shots	4.375	4.5
Mixed method	4.465	4.44
Random shots	4.42	4.41
No shots	4.365	

Table 7: The highest score under 2 and 5 shots for all methods

Rank	Demonstration	Points
1	Prompt-M+ Max Entropy Shots(5Shots)	4.5
2	Prompt-K+ Most Helpful Shots(5Shots)	4.49
3	Prompt-D+ Most Helpful Shots(5Shots)	4.47
4	Prompt-D+ Mixed Method(5Shots)	4.465
5	Prompt-O+ Most Helpful Shots(5Shots)	4.44

Table 8: The best 5 demonstrations and corresponding scores

Table (7) shows that the score of language expression ability of different demonstrations is between 4 and 5, indicating that the generated problem is semantically close to the preset question. Without the addition of shots, the score is 4.365. When using demonstrations as inputs, the overall score is slightly higher than the score achieved without shots.

5.3 The impact of different shots on model output results

In the above exploration, we obtained results for 2-shots and 5-shots configurations, leading us to speculate that varying the number of shots could impact the generated questions. We used one of the better-performing demonstrations showcased in the data above: the combination of Prompt-M and Max Entropy shots(5shots). We aim to investigate how the number of shots included in the prompt design affects the quality and relevance of the questions generated by the model.

Number Of Shots	Prompt-M+Max Entropy			
	MaxSimilarity		GPT-4 Score	
1	0.7941	0.7722	4.34	4.28
2	0.8093	0.8040	4.29	4.29
3	0.8245	0.7901	4.36	4.22
4	0.8284	0.7858	4.41	4.34
5	0.8235	0.8112	4.46	4.51
6	0.8246	0.8314	4.44	4.39
7	0.8305	0.8081	4.39	4.17
8	0.8235	0.812	4.34	4.43
9	0.7917	0.8176	4.36	4.4
10	0.8022	0.7922	4.29	4.34

Table 9: Evaluation results for different quantities of shots

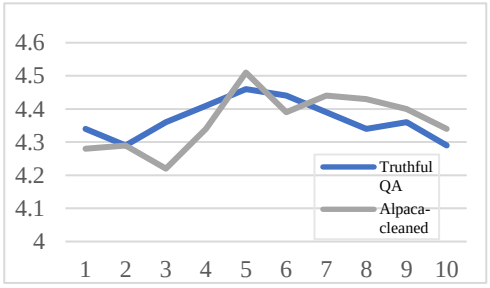


Figure 2: Cosine similarity in different shots.

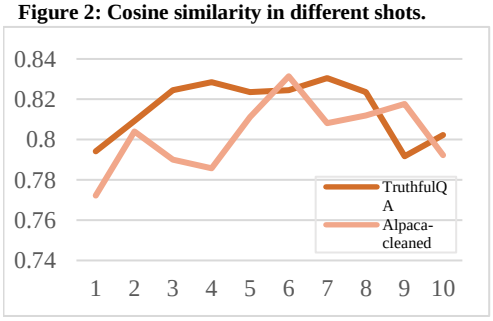


Figure 3: GPT scoring on different shots

The figure indicates that the number of shots has a significant impact on the quality of the questions generated by the model, with the overall trend showing an initial rise to a peak followed by a decline. In the evaluation results of cosine similarity, the best results were obtained from the TruthfulQA dataset with 7 shots, reaching 0.8305, and the Alpaca dataset with 6 shots, reaching 0.8314. The optimal number of shots for GPT-4 scoring results is 5, with an average score of 4.435 or above on both datasets. The results with around 1 to 3 shots are generally worse than those with around 5-7 shots. However, as the number of shots continued to increase beyond this range, the results did not improve but instead declined. It has been proven that adopting an appropriate number of shots in the demonstration can have a positive impact on generating questions.

5 Conclusion

In this paper, we propose methods for enhancing the quality of generated questions of large language models in the context of question generation. These methods include automatically generating prompts to form the model's input set, along with three demonstration construction methods: Most Helpful Shots, Maximum entropy Shots, and Mixed method. We use the GPT-3.5-turbo large language model and selected two datasets :TruthfulQA and Alpaca-cleaned.

When evaluating the quality of generated questions, we used cosine similarity to measure the semantic gap between the generated questions and the preset questions. Additionally, we employed the GPT-4 large language model, which has a stronger understanding capability than GPT-3.5-turbo, to score the generated questions. Among the results of cosine text similarity and GPT-4 score, one of the best demonstrations is the combination of prompt-F and 5 Max entropy shots. The content of prompt-F is "***Create 5 queries by extracting information from the text.***", that compared with baseline (Prompt-A: " ***Generate 5 questions from the given text.***", no shots), the text similarity and GPT-4 scores improved by 0.0309 and 0.195 respectively under two evaluation methods, proving the effectiveness of the demonstration construction method in improving the quality of generated problems.

References

[1] Kurdi, G., Leo, J., Parsia, B. *et al.* A Systematic Review of Automatic Question Generation for Educational Purposes. *Int J Artif Intell Educ* 30, 121–204(2020).Available: https://doi.org/10.1007/s40593-019-00186-y

[2] Jules White, Quchen Fu, Sam Hays, Michael Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith and Douglas C. Schmidt.2023. A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT. arXiv:2302.11382 [cs.SE]

[3] Andrew NG and Isa Fulford. Chatgpt prompt engineering for developers, short course. Available: https://www.deeplearning.ai/short-courses/chatgpt-prompt-engineering-for-developers/.

[4] chmidt, D. C., Spencer-Smith, J., Fu, Q., & White, J. Cataloging Prompt Patterns to Enhance the Discipline of Prompt Engineering. 2023.Available: https://www.dre.vanderbilt.edu/~schmidt/PDF/ADA_Europe_Position_Paper.

[5] Meskó B.Prompt Engineering as an Important Emerging Skill for Medical Professionals: Tutorial. J Med Internet Res 2023;25:e50638 .Available:https://www.jmir.org/2023/1/e50638 DOI: 10.2196/50638

[6] Rubaba Amyeen.2023.Prompt-Engineering and Transformer-based Question Generation and Evaluation. arXiv:2310.18867 [cs.CL]

[7] Stephanie Lin, Jacob Hilton and Owain Evans.2022.TruthfulQA: Measuring How Models Mimic Human Falsehoods. arXiv:2109.07958 [cs.CL]

[8] R. Taori, I. Gulrajani, T. Zhang, Y. Dubois, X. Li, C. Guestrin, P. Liang and T. B. Hashimoto. Stanford Alpaca: An Instruction-following LLaMA model, 2023.Available:https://github.com/tatsu-lab/stanford_alpaca